

Analysis: Go, Gophers!

Let $M = 25$ be the maximum possible number of gophers.

Test set 1

Our approach to test set 1 will be as follows: first, we will find the minimum taste level among all gophers, and then we will use it to determine the total number of gophers N .

To find the minimum taste level, we can simply [binary search](#) using the question "is the minimum taste level strictly greater than X ?" This is equivalent to asking: "Would a snack of quality X go uneaten by all gophers?" To answer such a question, we can offer $2M-1$ snacks of quality X consecutively, which guarantees that each gopher is exposed to at least one of them. If none of those snacks are eaten, every gopher's taste level must be greater than X , but if at least one snack is eaten, then there is at least one gopher with taste level at or below X . We can even stop offering snacks as soon as one gets eaten, if we want to save some snacks. This requires $\text{ceil}(\log 10^6) \times 49 = 980$ snacks at the most.

Once we know that there is a gopher g with level exactly L , and that L is the minimum taste level, we can use our snacks to answer a query "is it g 's turn?" by offering a snack of quality L , because g is the only gopher who would eat it. If we offer that many times in a row and calculate the fraction of eaten snacks, that should approximate $1/\text{number of gophers}$ fairly well. At this point, we might as well use our enormous number of leftover snacks to estimate, and then just answer with the N such that the result is closest to $1/N$. It turns out that M^3 tries of this experiment guarantee that the answer is correct, even in the worst case. The proof is included below as part of the explanation of the solution for test set 2.

Test set 2

The solution from test set 1 does not extend naturally to test set 2. In particular, it no longer suffices to find out what fraction of gophers have the minimum (or maximum) taste level, because there could be multiple gophers with that taste level; if we find a fraction of $1/K$, the number of gophers could be any multiple of K . So, we need to investigate other levels. Investigating taste levels other than an extreme one brings about the problem of the result being impacted by gophers with taste levels other than the one we are interested in. We can still use the idea of answering general queries by repeating snacks of the same quality, but they are significantly more complicated than a simple disjunction of the snack outcomes.

The first query Q we describe is somewhat natural: What is the exact fraction of gophers with taste level $\geq X$?

Notice that this query alone is enough to solve the problem: we can do a binary search of sorts: given a range $[A, B]$ of at least two levels, and known fractions of gophers of those taste levels X and Y , respectively, we can calculate the fraction of gophers Z for taste level $C = (A + B) / 2$. We recurse over the range $[A, C]$ if $X \neq Z$ and over the range $[C, B]$ if $Y \neq Z$, because the fractions are different if and only if there is a gopher with a taste level in the range producing the change. This algorithm allows to identify all levels at which at least one gopher exists. For each of them we calculate the fraction of gophers at level L using our query and then subtracting the fraction of gophers at each other level $< L$. Finally, we can take the [least common multiple](#) (LCM) of the denominators of all those fractions to find the answer.

This algorithm requires about $N \times \text{ceil}(\log 10^6 - N)$ queries like Q to be made. Unfortunately, we see below that this would be too many.

One way W1 of solving Q is to try enough snacks and then round to the nearest feasible fraction. If we use enough snacks, the rounding will give the exact answer. Note that if we give X consecutive snacks, we are guaranteed to have the right answer in all but a prefix and a suffix of the tries, both of which have length up to M-1. This bounds the error by $2M-2$. Moreover, since our experiment has a binary answer, the farthest we can be from the true number of positive (or negative) answers is M-1, in the case when there are exactly $\text{ceil}(M/2)$ answers of one type and $\text{floor}(M/2)$ of the other, and we happen to hit both a prefix and suffix of size $\text{floor}(M/2)$ giving the same answer (this is for odd M, the worst case would be M for an even M).

Additionally, since we only consider fractions with denominators up to M, the distance between two different results is bounded by $1 / (M \times (M-1))$. This means that if our experiment has an error of less than half of that, rounding is guaranteed to produce the right answer. Putting the total error of M together, this implies that we need M^3 snacks to get a perfect answer for Q. A total number of snacks of $M \times \text{ceil}(\log 10^6 - M) \times M^3$ exceeds the allotted total by a large margin.

A different way W2 to answer Q is to always use R consecutive snacks, where R is the LCM of all the possible results. That means the error is always zero, since we give each gopher exactly the same number of snacks. Unfortunately, $\text{LCM}(2,3,\dots,25)$ is also much too large, so this doesn't work either.

Another strategy to reduce the number of queries is to notice that we only need an exact number for the final levels, right before doing the LCM to get the result. For all intermediate values of our "multi-leaf binary search", we only need to decide whether there is some gopher in a range or not. One way W3 to do it would be to, instead of using exact fractions X, Y and Z for A, B and C above, have approximations X', Y' and Z' that are good enough that we can decide whether the real numbers are equal. Using $2M^2$ snacks guarantees that we will encounter each gopher at least 2M times, which means that if X and Z are different, their approximations of the number of total positives will differ by at least 2M. Since we showed that the total error of both approximations is at most M-1, the error of the difference is at most $2M-1$, which means comparing that difference with 2M is enough to determine whether the real fractions are equal or not. This significantly reduces the total required number of snacks, since we only need to use $M \times \text{ceil}(\log 10^6 - M) \times M^2$ for the multi-leaf binary search, and then $M \times M^3$ or $M \times R$ to get the final precise answers. However, this is still too many.

The final algorithm requires us to use all 3 of the variants above: we start the multi-leaf binary search using W3. Each time we find a level, we use either W1 or W2, whichever is cheapest, to get the real fraction of the found level. If we recurse on the larger intervals first, we'll find the levels from highest to lowest, so we can do the subtraction. Once we have the real fraction, we potentially restrict the number of possible results N to the multiples of the denominator. This reduces R. Eventually, R might be small enough that we can use W2 for the binary search instead of W3 as well. Notice that each time we use W2, since we started with other methods, we need an additional R to "align" ourselves, depending on how many snacks we have used so far. However, since we eventually use R for everything, the additional cost of these alignments is extremely small.

We leave the precise analysis of total number of snacks needed to the reader, but it's possible, with some careful bounding, to prove that it never exceeds **S**, and we couldn't find a case that gets above ~85% of **S**. The reason is that if the denominator of a fraction is larger than M/2, then there's only one possible result left and we are done. If it's less than M/2 but somewhat large, the number of possibilities for the result is reduced significantly, and its LCM is reduced even more because the GCD of those possibilities is more than 1. If the denominator is small, it means that the found level has multiple gophers, which reduces the total cost of the multi-leaf binary search.

Analysis: Jurisdiction Restrictions

Test set 1

We begin modeling the problem as a [flow network](#). We construct a directed graph G where the set of nodes is $\{\text{source}, \text{sink}\} \cup S \cup B$ where S is the set of blocks that contain a station and B is the set of blocks that do not contain a station and are within reach of some station. The edges of the graph are $\{\text{source} \rightarrow s : s \in S\} \cup \{s \rightarrow b : s \in S, b \in B\} \cup \{b \rightarrow \text{sink} : b \in B\}$. Notice that each path from source to sink links a specific station with a specific block. If we refine G by assign capacity $|B|$ to each edge $\text{source} \rightarrow s$, and capacity 1 to each other edge, a valid flow of the network is a union of paths from source to sink, and due to the capacities, each station will belong to at most one of those paths. Moreover, a maximum flow will cover all stations, so there is a bijection between maximum flows and station assignments. In this way, we can reframe the problem as minimizing the difference between maximum and minimum flows going through edges that come out of the source in a maximum flow of G .

If we want to set an upper bound U for the flow going through edges coming out of the source, we can adjust the capacities of those edges to U . If we want to set a lower bound L on those quantities, we can add a second copy of each of those edges with capacity L and have a cost of 0 in the new edges and 1 in the max-capacity version (plus, cost 0 for all other types of edges). Using [min-cost max-flow](#) instead of max flow will give preference to flows that use at least L capacity through each station, and if the retrieved flow doesn't use the full capacity, it means that one doesn't exist.

With the above, we can just try every possibility for L and U , discarding the combinations that don't achieve a max flow of $|B|$ that saturates all the cost 0 edges coming out of the source. The non-discarded combination with minimum $U-L$ is the answer. Both U and L are bounded by $|B|$, which is bounded by $\mathbf{R} \times \mathbf{C}$. The size of G is $O(\mathbf{S} \times \mathbf{R} \times \mathbf{C})$, and min-cost max-flow on a graph with only 0 and 1 costs takes only linear time per augmenting path, so quadratic time overall.

This means the described algorithm takes time $O(\mathbf{S}^2 \times \mathbf{R}^4 \times \mathbf{C}^4)$. This can be too slow even for test set 1, but there are some optimizations that will shed some of that time, and you only need one of them to pass test set 1.

- Instead of trying all combinations of L and U , you can try only the ones that have a chance to improve: if a combination (L, U) doesn't work, then we know $(L+1, U)$, $(L+2, U)$, etc will also not work, so we can move on to $(L, U+1)$. If a combination (L, U) works, we know that $(L, U+1)$, $(L, U+2)$, etc will also work but will yield a larger difference, so we can skip them and move directly to $(L+1, U)$. This requires us to test a linear (instead of quadratic) number of combinations of L and U .
- An even simpler version of the optimization above is to try all possibilities for only one of L or U , and then use binary search to find the optimal choice for the other. This doesn't take the number of combinations down to linear in $|B|$, but it does make it [quasilinear](#).
- A flow for the combination (L, U) is also a valid flow for the combination $(L, U+1)$, so instead of starting the flow from scratch, we can use it, dramatically reducing the number of augmenting paths we need to find overall.

Test set 2

For test set 2, the values of $\mathbf{R}$ and $\mathbf{C}$ are too high to appear linearly in the running time of the algorithm. That means we have to improve upon the solution above in at least two fronts: the size of G , which is linear on $\mathbf{R}$ and $\mathbf{C}$ and ultimately impacts any algorithm we want to run on G , and the number of combinations of L and U we try, which is also linear or more.

We start by defining G for test set 2 a little differently, using a technique commonly known as coordinate compression. We define B' as a set of sets of blocks that form a partition of B above. The way to partition is by taking the full grid and cutting through up to $2S$ horizontal and $2S$ vertical lines at the boundaries of the jurisdiction of each station. The resulting rectangles have the property that the set of stations that can reach any block of the rectangle is the same. Rectangles only represent the blocks within them that do not contain a station. We can now define G as having nodes $\{\text{source}, \text{sink}\} \cup S \cup B'$ and edges $\{\text{source} \rightarrow s : s \in S\} \cup \{s \rightarrow b' : s \in S, b' \in B'\} \cup \{b' \rightarrow \text{sink} : b' \in B'\}$. Note that each node in B' now represents a set of blocks, so we also have to fiddle with capacities. Edges $\text{source} \rightarrow s$ have capacity $|B|$ (effectively infinity) as before, and so do edges $s \rightarrow b'$. Edges $b' \rightarrow \text{sink}$, however, have capacity $|b'|$. In this way, each node b' is the union of many nodes b from the previous version of G , so its outgoing edge has the sum of the capacities of the outgoing edges from those nodes b . The size of this G is $O(S^3)$ with $O(S^2)$ nodes.

Now, in G , each path $\text{source} \rightarrow s \rightarrow b' \rightarrow \text{sink}$ that carries flow X represents that X blocks from set b' are assigned to station s . Notice that there is no longer a bijection between flows and station assignments, but there is a bijection if we consider permutations of blocks within a rectangle to be equivalent. Since all those blocks are equivalent for the purpose of this problem, this bijection is enough. As before, we now have to find a maximum flow in G such that the difference between the maximum and minimum flow in edges going out of the sink is minimized.

Since we cannot try all possibilities for L and U , we will use a little theory. Let G_C be a copy of G , but with the capacities of all edges $\text{source} \rightarrow s$ changed to C . $G = G_{|B|}$. Notice that a valid flow in G_C is also a valid flow in any $G_{C'}$ with $C' \geq C$.

Now, let U be minimum such that G_U allows a flow of total size $|B|$. Let L be maximum such that there exists a maximum flow of size $|S| \times L$ in G_L , that is, there is a flow that saturates all edges of type $\text{source} \rightarrow s$. Any maximum flow in G has at least one edge of type $\text{source} \rightarrow \text{sink}$ carrying flow at least U (otherwise, such maximum flow is a valid flow in G_{U-1} , contradicting the definition of U). Also, any maximum flow in G has at least one edge of type $\text{source} \rightarrow \text{sink}$ carrying at most L (otherwise, we can subtract flow from any path sink to source that carries more than $L+1$ to obtain a flow that carries exactly $L+1$ through all nodes, and thus it would be a valid flow in G_{L+1} of size $|S| \times (L+1)$, contradicting the definition of L). Finally, if we start with the flow in G_L that justifies the definition of L and use it as a valid flow in G_U , we know that it can be extended to a maximum flow F in G_U via augmenting paths. Since the augmenting paths don't contain cycles, they don't decrease flow in edges of type $\text{source} \rightarrow s$. Therefore, F is a maximum flow in G_U such that the difference between the maximum and minimum flow going through edges coming out of the source is exactly $U - L$. And by definition, a maximum flow of G_U is a maximum flow of G . The previous observations show that there is no flow in G with such a difference less than $U - L$, so $U - L$ is the answer to the problem. We can binary search for L and U independently using their original definitions, which yields an algorithm that takes time $O(S^5 \times \log(R \times C))$.

Analysis: Swordmaster

The only way we can fail to become the Swordmaster is to get stuck in a situation where for all remaining duelists (duelists that we have not beaten yet) we will not defeat them without learning additional skills. At any point in time, the remaining duelists can be divided into four groups:

1. D1: Duelists we certainly can defeat.
2. D2: Duelists that have an attack we cannot defend against and a defense for every one of our attacks.
3. D3: Duelists that do not have a defense for one of our attacks, but have an attack we cannot defend against.
4. D4: Duelists that do not have an attack we cannot defend against, but have a defense for every one of our attacks.

If there is any remaining duelist in D1, we should just defeat them. If there is someone in D3 and there is someone in D4, then we can make some progress (i.e. reducing the number of remaining duelists, or moving a remaining duelist to D1) by doing the following:

1. We choose any duelist in D3 (call them A) and any duelist in D4 (call them B).
2. We ask A for a duel. Since A is in D3, A has an attack that we don't defend against. Either A will use an attack that we don't defend against (thus we will learn the attack), or we can defend and win the duel (thus learning all of A's attacks). At the end of the duel, we will learn an attack which we can't defend against. Let us call this attack is a .
3. If B cannot defend against a , then B has moved to D1 and we have made some progress. Otherwise, we ask B for a duel and use attack a . If B cannot defend, then we will win the duel and make some progress. Otherwise, we will learn defense a .
4. Now that we have learned defense a , A might have moved to D1, or might still be in D3 (by having another attack that we can't defend against). If A moved to D1, then we have made progress. Otherwise, we repeat step 2 until A has no attack that we can't defend against.

Therefore, the only possibilities in which we might fail to become the Swordmaster are when either every remaining duelist is in D2 or D3, or every remaining duelist is in D2 or D4.

If every remaining duelist is in D2 or D3, then every remaining duelist has an attack we can't defend. From this point, every remaining duelist can just attack us with an attack we can't defend against and choose to not defend against, which means that we will not learn any new defenses. Therefore, we can't defeat any more duelists and are certain to fail to become the Swordmaster. Notice that we can actually detect this situation up front — if there exists a group of duelists G_1 (not containing us), such that every duelist in G_1 has at least one attack against which nobody outside G_1 can defend, then we are doomed. The strategy for the duelists is that everybody in G_1 can just successfully attack us and choose to not defend (thus we are not learning any new defense known by duelists in G_1).

We can check the existence of G_1 by starting with G_1' , the set of all duelists except us. We can iteratively remove from G_1' any duelist we can defend against regardless of their attack, and get all their defenses. This can be done in $O(N \times P)$ time. This complexity is also bounded by the sum of the number of attacks and defenses known to all duelists, which is linear in the size of the input.

Test set 1

For test set 1, the situation when every remaining duelist is in D2 or D4 is similar. This means every remaining duelist can defend against all of our attacks. Therefore, there exists a group of duelists G_2 (not containing us), such that every duelist in G_2 has a defense for every attack available outside G_2 . Therefore, we are also doomed, since everybody in G_2 can just defend against our attack and always use attack 1 (thus we are not learning any new attack known by duelists in G_2). We can check the existence of such G_2 up front with an algorithm analogous to the one presented above to check for G_1 .

Test set 2

For test set 2, the situation when every remaining duelist is in D2 or D4 is complex. The difference is that our opponents **have to** attack, so even when dueling with some duelist who can defend against all of our attacks, we can learn a new attack. This new attack potentially causes some duelist to move from D4 to D1, or from D2 to D3. Therefore, the fact that every remaining duelist is in D2 or D4 is not a sufficient condition for us to fail to become the Swordmaster.

Now imagine we are in a situation where we are actually doomed — we are in a position where, finally, every remaining duelist is in D2 or D4 and we will never defeat any duelist, nor will learn a new attack or defense. Such a situation has to be reached at some point since there is only a finite number of steps of learning (attack or defense) to be made. In this situation, we could fight a duel with every remaining duelist and learn the attacks they use. By definition, we already knew all these attacks, and so, also by definition, all our opponents can defend against all these attacks. So, for us to be doomed through this path, there must exist a group G_2 of duelists and an assignment of an attack for each of them $a : G_2 \rightarrow \text{Attacks}$, such that every duelist d in G_2 knows attack $a(d)$ and knows how to defend against every attack known outside G_2 and every attack $a(d')$ for d' in G_2 .

This condition is harder to check. Let us assume there exists such G_2 . Consider any two duelists X and Y . If X has an attack that Y cannot defend against, then Y being in G_2 implies X must be in G_2 as well. This defines a directed graph on the duelists, with the edge going from Y to X .

We find the [strongly connected components](#) (SCCs) of this graph. If any duelist of a SCC is in G_2 , then the whole SCC is in G_2 . Therefore, G_2 will be an upper subset (a subset with no edges going out of the subset) of the [directed acyclic graph](#) (DAG) of the strongly connected components of the graph. Notice that if we can choose some upper subset of duelists to be a valid set G_2 , then any subset of G_2 which is also an upper subset will also be a valid G_2 with the same attack selections. Therefore, it is enough to check only leaf SCCs in the DAG. If any valid G_2 exists, then a G_2 which is a leaf in the DAG also exists.

Therefore, for each leaf SCC in the DAG, we can check whether each duelist in the SCC can choose an attack that every duelist in the SCC can defend against. To do this, we can first find the set D which is an intersection of the defenses known by every duelist in the SCC, and see whether every duelist in the SCC has an attack in D .

To construct the graph in $O(N \times P)$ time, we can introduce nodes for attacks as well. We add an edge from a duelist to an attack if the duelist cannot defend against the attack, and an edge from an attack to a duelist if the duelist knows that attack. The overall solution runs in $O(N \times P)$ time.

Analysis: The Cartesian Job

The first thing to notice about the problem is that all lasers rotate at the same speed. That means that after each second, regardless of direction, the lasers will be back at the configuration given in the input, and each subsequent second will just be a copy of the previous one. Then, we only need to check what happens within a second. In what follows, we assume the configuration given in the input happens at time = 0 seconds and we only care about what happens for times in the range 0 to 1 seconds.

Test set 1

In Test set 1 there are such a low number of lasers that we can try every possible combination of directions and add 2^{-N} to the result for each one that leaves the segment uncovered some of the time.

For fixed directions of rotation, we can check whether the segment is covered by first mapping each laser to the interval of time during which it will cover the segment and then seeing if the union of all those intervals covers the full second. We refer to intervals in the modulo 1-second ring; that is, an interval can start at t_0 and end at $t_1 < t_0$, covering the time from t_0 to 1 second and from 0 seconds to t_1 . There are several ways to check if the union of intervals, even intervals in a ring, cover the full second, but a simple one is to split the intervals that wrap around into two non-wrapping-around ones, sort those intervals by start time, let $t = 0$, and do for each interval (t_0, t_1) , if $t_0 > t$, then there's a hole; else, $t = \max(t, t_1)$. If we reach the end, there's a hole if and only if $t < 1$.

To obtain the covering interval for a given laser with endpoint p and second point q we check the angle $qp(0,0)$ and $qp(0,1000)$, and then divide by 2π . Notice that if we do this with floating point arithmetic, it is highly likely that we will have precision problems. We can do it with integer only arithmetic by representing the times symbolically as the angles represented by the original vectors. To simplify the presentation, we assume we computed actual times, but all the operations needed for the rest of the solution can be implemented for an indirect representation of those times intervals.

Since we need to check every combination of clockwise and counterclockwise rotations, and evaluating a particular combination requires looping over each laser, this solution takes $O(N \times 2^N)$ time, which is sufficient for Test set 1.

Test set 2

We start by observing that the two time intervals corresponding to each direction of rotation of a given laser are symmetrical; that is, they are of the form (t_0, t_1) and $(1-t_1, 1-t_0)$. Notice that the symmetry is both to the 1/2 second point and to the extreme 0 seconds / 1 second, because we are in a ring. Adding the fact that intervals are less than 1/2 long, we can notice that the pair of intervals coming from a given laser can be categorized into one of three types:

- Two non-overlapping intervals, one within $(0, 1/2)$ and the other within $(1/2, 1)$. Example: $[0.2, 0.3]$ and $[0.7, 0.8]$.
- Two intervals that overlap around 1/2. Example: $[0.3, 0.6]$ and $[0.4, 0.7]$.
- Two intervals that overlap around 0 a.k.a. 1. Example: $[0.8, 0.1]$ and $[0.9, 0.2]$.

For the last two types, the overlapped area is an interval of time during which the segment is guaranteed to be guarded. Therefore, we can remove the overlapped part from further consideration and assume that the two intervals are just their symmetric difference. In the first example above, we'd remove $[0.4, 0.6]$ from consideration and keep the pair of intervals $[0.3, 0.4]$ and $[0.6, 0.7]$. After we do this with all intervals, the part that remains to be considered is some subinterval of $(0, 1/2)$ and some subinterval of $(1/2, 1)$, and each pair of intervals is exactly two symmetrical intervals, one on each side.

At this point the problem we need to solve is, given two symmetrical intervals u_1 within $(0, 1/2)$ and u_2 within $(1/2, 1)$, and a set of pairs of intervals (a, b) , $(1 - b, 1 - a)$, what is the probability that the union of picking one from every pair uniformly at random does not cover both u_1 and u_2 ? Notice that because of symmetry, one particular split (a collection of one interval from each pair) covers u_1 if and only if the opposite split covers u_2 . So, an alternate way to frame the problem is: given the list of intervals from each pair that are inside u_1 , what is the probability at least one part of a random split of them doesn't cover u_1 ? This problem we can solve with a dynamic programming approach over the list of intervals.

The dynamic programming algorithm is, in a way, a simulation of the algorithm we presented above to check if the union of the intervals covers the universe, over all combinations at once. We iterate the intervals in non-decreasing order of left end. We also keep a state of which prefix of u_1 each side of the split has already covered. Formally, if $u_1 = (v, w)$ we calculate a function $f(i, x, y) = \text{probability of a split of intervals } i, i+1, \dots, N \text{ not covering both the remainder of the first side } (x, w) \text{ and the remainder of the second side } (y, w)$. Equivalently, this is the probability of the cover not being full given that intervals $1, 2, \dots, i$ are split in a way that the union of the intervals from one side of the split is (v, x) and the union of the intervals from the other side is (v, y) . The base case is if $\min(x, y) \geq w$, then $f(i, x, y) = 0$, or if $i > N$ or $\min(x, y) > \text{the left end of the } i\text{-th interval}$, then $f(i, x, y) = 1$. For all other cases, we can calculate $f(i, x, y) = (f(i+1, \max(x, b), y) + f(i+1, x, \max(y, b))) / 2$, where (a, b) is the i -th interval in the order. Finally, the answer to the problem is $f(0, v, v)$.

Noticing the values x and y for which we need to calculate f are always either s or the right end of an interval bounds the size of the part of f 's domain that we need, and thus the running time of the algorithm, by $O(N^3)$. We can further notice that $\max(x, y)$ is always equal to the maximum between s and the maximum right end of intervals $0, 1, \dots, i-1$. This reduces the domain size and running time to $O(N^2)$. One further optimization needed is to notice that if $\min(x, y)$ is not one of the largest K right ends of intervals $0, 1, \dots, i-1$, then the result of $f(i, x, y)$ is multiplied by 2^{-K} or less to calculate the final answer. For values as small as 50 for K , that means the impact in the answer of the value of f is negligible in those cases and we can just approximate it as 0, making the size of the domain of f we have to recursively calculate only $O(K \times N)$.

Implementing the dynamic programming algorithm as explained above can be tricky, especially if you want to memoize using integer indices over the intervals and largest K last values, as opposed to floating point numbers. However, doing a forward-looking iterative algorithm can be a lot easier. We maintain a dictionary of states to probability, where a state is just the two values x and y , always sorted so that $x \leq y$. We start with just $\{(s, s): 1\}$. Then, we consider each interval (a, b) iterative and for each state (s_1, s_2) with probability p in the last step's result, we add probability $p / 2$ to new states $\text{sorted}(\max(s_1, b), s_2)$ and $\text{sorted}(s_1, \max(s_2, b))$ if $a \leq s_1$. If $a > s_1$, we add p to our accumulated result and don't generate any new state, since state (s_1, s_2) is guaranteed to leave some unguarded time. This is a simple implementation of the quadratic version, but the real trick is when making the optimization to bring the time down to linear, which in this case is simply ignoring states with too low probability (i.e., if $p < \epsilon$, do nothing).

Analysis: Two-Tiling

The restriction on tile size limits the problem to just 35 distinct tiles, and therefore "only" $35 * 34 / 2 = 595$ possible cases. We might consider whittling away at this number via individual insights — e.g., any case with a 9-cell tile and an 8-cell tile must be impossible, since the least common multiple of 9 and 8 is 72, which is larger than the allowed board size. We may even be tempted to cut out some paper tiles and do some solving by hand, but if so, we will quickly discover that this is harder than it seems, and it is particularly hard to be sure that a case is impossible. We need to either write a Master's thesis on properties of n-ominoes, or take an exhaustive approach.

We cannot simply find all of the ways to tile part or all of the board with each tile, and then intersect those two sets; there could be too many possible tilings to generate (e.g., 2^{64} for the 1x1 tile). Even if we could come up with special-case solutions to get around this issue for enough of the smaller tiles, we would still waste a lot of time enumerating sets that are obviously incompatible with the other tile.

Instead, we can use a backtracking strategy that takes advantage of our constraints, using the placement one type of tile to guide how we place the other type. The strategy does not have to be lightning-fast; we know that every case will appear in the test set, so we can enumerate and solve them all offline and then submit once we have an answer for each one. However, it must be fast enough to generate all 595 answers before the World Finals end!

Let us number the cells of the 8x8 board in row-major order, which is the same order in which our backtracking search will explore. We can translate each tile (and all of its rotations and reflections) around the 8x8 board to exhaustively find all possible placements. Then, we can assign each of those placements to the lowest-numbered cell that the placement uses. For example, one placement of the 2x2 square tile can cover cells 1, 2, 9, and 10 of the board, and the lowest-numbered of those cells is 1, so we assign that placement to cell 1. Now we have a helpful map from each board cell to the tile placements that fit there. Notice that it would be redundant to list our example 2x2 square tile placement under cell 10. For example, by the time our search reaches cell 10, it will have already considered that same placement at cell 1, so there would be no need to consider it again.

Then, starting in the upper left cell and going in row-major order, we try to build a set of cells that can be tiled by both tiles. For each cell c , we decide whether to fill it. If it is already covered by a previous tile placement, we must fill it; if we decide to fill it, and it is not already covered by one or both types of tiles, then we have our search explore all of the tile positions assigned to that cell in our map above.

Since the board is 8x8, we can use the bits of a 64-bit data type to represent each tile's coverage of the board, and we can use bitmasking to quickly see whether desired cells are occupied, or generate one state from an existing state. It is also very quick to check whether two states are equal; if this is ever the case, we stop immediately, step back through our backtracking to figure out exactly how to divide that set up into tiles of each type, and output that tiling. If the backtracking process finishes without finding such a set, then the case is impossible.

This solution turns out to be fast in practice, running in seconds. Intuitively, small tiles are more "flexible" and it is easier to find solutions quickly for cases involving them, whereas large tiles fill up the board's cells rapidly and do not open up as many choices. That is, the recursion tree for the backtracking is wide but short in the case of small tiles, and tall but narrow in the case of large tiles.

This approach does not necessarily find minimal sets of cells, but it can generate some artistically pleasing shapes and tilings, and/or puzzles that really would be fun to solve! Here are a couple examples generated by one of our solutions:

These angular tiles create a solution with a surprising mix of dense and holey areas!

```
aaa.bbb. AAA.BBB.  
a..cbcb. A..CCCB.  
aaacbcbb. ADDDECB.  
...ccc.. ...DEC..  
..dddeee ..FDEEEG  
..d.....e ..F....G  
..dddeee ..FFFGGG  
.....
```

We hope you "loved" this problem! Notice that a test case is not necessarily any easier when the two tiles differ in only a very small way.

```
aa...bb. AA...BB.  
aaa..bbb AAA..CBB  
acacdbdb DDACCCBB  
.cccddd. .DDCCEE.  
.cceedd. .DDFFEE.  
..eefff. ..FFEEG.  
..eeeff. ..FFGGG.  
....ff.. ....GG..
```